Министерство науки и высшего образования Российской Федерации

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Мастер-класс № 3

«Программирование микроконтроллеров с операционной системой (FreeRTOS)»

Вариант: Flappy Bird

Исполнители: Ясаков Артем
Пахомов Игнат

Преподаватели: Быковский С. В.
Гончаров А. А.

Санкт-Петербург
2026

Contents

1 Цель и задачи работы **2**

2 Схема взаимодействия процессора и периферийных устройств **2**

3 Описание процессов, изображённых на схеме **4**

3.1 Инициализация 4

3.2 Подсистема ввода (Task_Input) 4

3.3 Подсистема игровой логики (Task_Game) 5

3.3.1 Генерация препятствий 5

3.4 Подсистема отображения (Task_Display) 5

3.5 Подсистема звука (Task_Sound) 6

3.6 Синхронизация и защита от гонок 6

3.7 Конфигурация FreeRTOS 7

4 Руководство пользователя **7**

4.1 Подключение стенда 7

4.2 Загрузка программы 7

4.3 Управление 8

4.4 Игровой процесс 8

4.5 Скриншоты 9

4.6 Тестовые сценарии 10

5 Ссылка на репозиторий с кодом **10**

1 Цель и задачи работы

Цель работы: получить практические навыки разработки многозадачных программ для микроконтроллеров с использованием операционной системы реального времени FreeRTOS, а также освоить механизмы межзадачного взаимодействия: очереди, мьютексы и планирование задач.

Задачи:

1. Изучить архитектуру FreeRTOS: задачи, планировщик, очереди, мьютексы, семафоры.
2. Освоить процесс создания проекта с FreeRTOS в среде STM32CubeIDE (CMSIS-RTOS v1).
3. Разработать многозадачное приложение на базе FreeRTOS: разбить функциональность игры Flappy Bird на независимые задачи.
4. Реализовать драйвер опроса клавиатуры через I2C (расширитель PCA9538) и драйвер вывода пиксельной графики на OLED-дисплей SSD1306.
5. Реализовать звуковое сопровождение через PWM (TIM2) на пьезо-зуммере.
6. Реализовать игровую механику Flappy Bird: гравитацию, генерацию препятствий, детектирование столкновений, подсчёт очков.
7. Обеспечить защиту от гонок данных с помощью мьютексов.
8. Протестировать программу на стенде SDK-1.1M.

2 Схема взаимодействия процессора и периферийных устройств

Схема взаимодействия выполнена в виде диаграммы последовательностей (PlantUML).

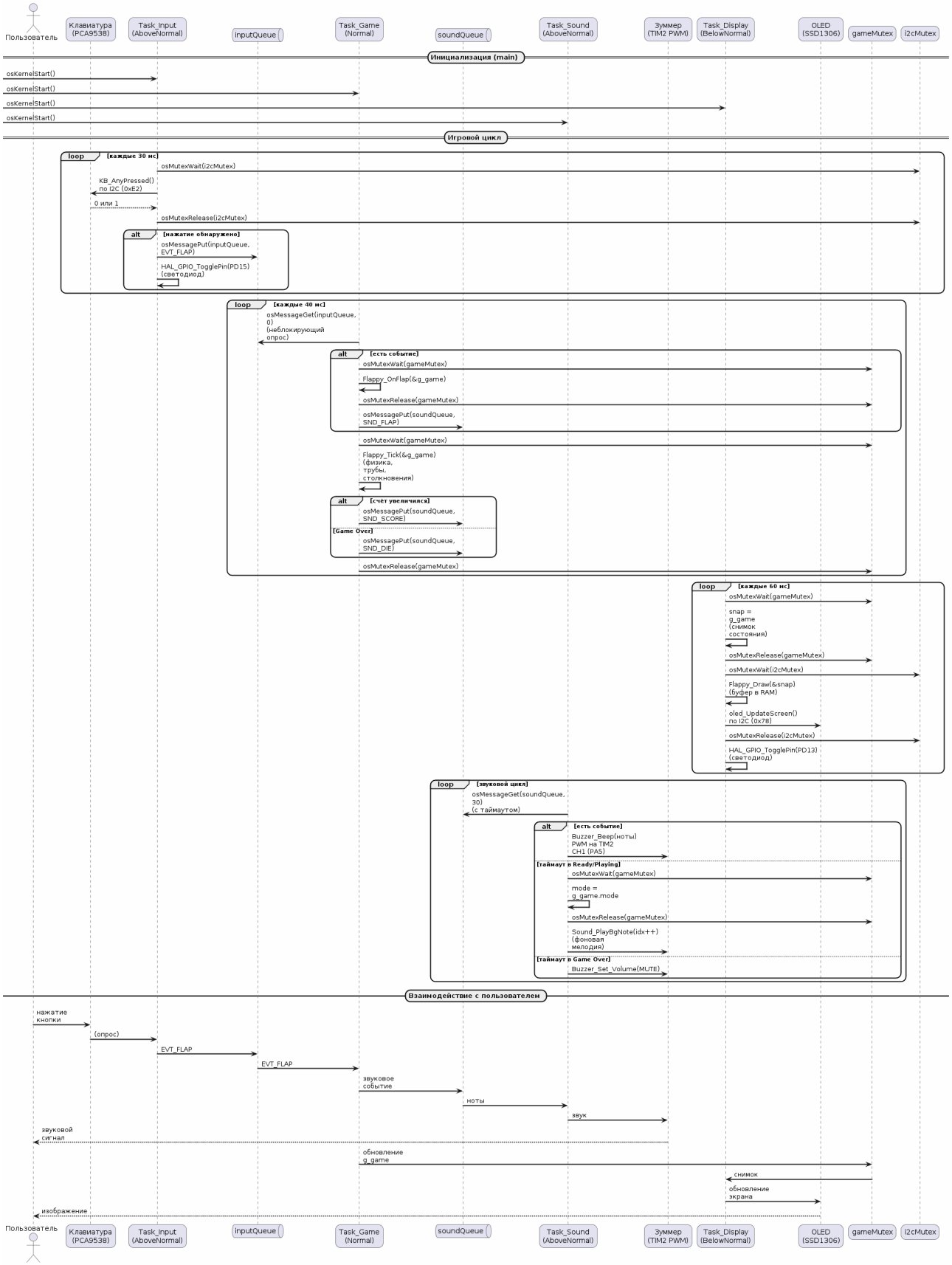


Figure 1: Диаграмма последовательности взаимодействия задач FreeRTOS и периферии

3 Описание процессов, изображённых на схеме

3.1 Инициализация

В функции `main()` выполняется:

- `HAL_Init()` — инициализация HAL и SysTick;
- `SystemClock_Config()` — HSE 25 МГц, PLL → SYSCLK 168 МГц, AHB = 168 МГц, APB1 = 42 МГц, APB2 = 84 МГц;
- `MX_GPIO_Init()`, `MX_I2C1_Init()`, `MX_USART1_UART_Init()`, `MX_TIM2_Init()` — настройка периферии (генерируется CubeMX);
- `oled_Init()` — отправка по I2C на адрес `0x78` команд инициализации SSD1306;
- `Buzzer_Init()` — инициализация ШИМ на TIM2 CH1 (PA5);
- `Buzzer_BootBeep()` — два звуковых сигнала при включении;
- `MX_FREERTOS_Init()` — создание очередей, мьютексов и задач FreeRTOS;
- `osKernelStart()` — запуск планировщика.

3.2 Подсистема ввода (Task_Input)

Задача `Task_Input` (приоритет AboveNormal, стек 256 байт) опрашивает клавиатуру:

- Клавиатура подключена к расширителю PCA9538 (I2C-адрес `0xE2`), организована в матрицу 4×3.
- Строки — $P0..P3$ расширителя; столбцы — $P4..P6$.
- Функция `KB_AnyPressed()` последовательно опрашивает все строки через `Check_Row()` и возвращает 1 при любом нажатии.
- Реализован программный антидребезг: обнаружение перехода «не нажато → нажато» (переменная `prev`).
- При обнаружении нажатия в очередь `inputQueue` отправляется событие `EVT_FLAP`.
- Индикация нажатия: светодиод PD15 переключается при каждом нажатии (`HAL_GPIO_TogglePi`
- Доступ к шине I2C защищён мьютексом `i2cMutex`.
- Период опроса: 30 мс.

3.3 Подсистема игровой логики (Task_Game)

Задача `Task_Game` (приоритет Normal, стек 256 байт) содержит основной игровой цикл:

- Считывает все накопившиеся события из `inputQueue` (неблокирующий `osMessageGet` с таймаутом 0).
- При получении `EVT_FLAP` вызывает `Flappy_OnFlap()`, которая в зависимости от режима: запускает игру (Ready), задаёт скорость взмаха (Playing) или перезапускает игру (Game Over).
- Каждый тик (40 мс) вызывает `Flappy_Tick()`:
  - Обновляет вертикальную скорость птицы с учётом гравитации (`GRAVITY = 18`);
  - Двигает трубы влево (`PIPE_SPEED = 2`);
  - Проверяет столкновения (`bird_hits_pipe`): перекрытие по X и выход за пределы зазора по Y;
  - Проверяет выход за границы экрана (земля `GROUND_Y = 60`);
  - При прохождении трубы увеличивает счёт и обновляет лучший результат (`best`);
  - Ушедшие за экран трубы пересоздаются справа со случайной позицией зазора.
- Отправляет звуковые события в `soundQueue` при изменении игрового состояния.
- Доступ к состоянию игры `g_game` защищён мьютексом `gameMutex`.

3.3.1 Генерация препятствий

Позиция зазора трубы вычисляется с помощью 16-битного LFSR-генератора псевдослучайных чисел:

```
lfsr ^= (lfsr << 7); lfsr ^= (lfsr >> 9); lfsr ^= (lfsr << 8);
```

Зазор располагается в диапазоне $[8, \texttt{SCREEN_H} - \texttt{PIPE_GAP} - 10]$, что обеспечивает проходимость уровня.

3.4 Подсистема отображения (Task_Display)

Задача `Task_Display` (приоритет BelowNormal, стек 768 байт) отрисовывает игру на OLED:

- Делает снимок состояния `g_game` под мьютексом `gameMutex` и работает с локальной копией — это исключает длительное удержание мьютекса во время отрисовки.
- Вызывает `Flappy_Draw()`, которая:
  - Заливает буфер чёрным цветом (`oled_Fill(Black)`);
  - Рисует землю (сплошной прямоугольник от `GROUND_Y` до низа экрана);

  - Рисует трубы (верхнюю и нижнюю части для каждой из двух труб) с помощью `fill_rect()`;
  - Рисует птицу (квадрат 6×6 пикселей с глазом);
  - Выводит счёт в правом верхнем углу;
  - В режиме Ready выводит надписи «FLAPPY» и «press key»;
  - В режиме Game Over выводит «OVER» и лучший результат.
- Отправляет кадровый буфер на дисплей через `oled_UpdateScreen()` (покадровая запись 8 страниц по I2C).
- Доступ к I2C защищён мьютексом `i2cMutex`.
- Индикация: светодиод PD13 переключается при каждой отрисовке.
- Период обновления: 60 мс.

3.5 Подсистема звука (Task_Sound)

Задача `Task_Sound` (приоритет AboveNormal, стек 384 байт) обеспечивает звуковое сопровождение:

- Приоритетно обрабатывает события из `soundQueue` (звуковые эффекты):
  - `SND_FLAP` — две короткие ноты C6 + E6;
  - `SND_SCORE` — три ноты E5 → G5 → C6;
  - `SND_DIE` — нисходящая мелодия из 7 нот (G5 → F4 → G3);
  - `SND_START` — восходящее арпеджио C5 → E5 → G5 → C6.
- При отсутствии событий в очереди (таймаут 30 мс) проигрывает фоновую мелодию — циклический мотив из 16 нот (C5, E5, G5, E5, C5, E5, G5, C6, A5, G5, E5, C5, D5, F5, A5, G5).
- В режиме Game Over зуммер отключается (`BUZZER_VOLUME_MUTE`).
- Для генерации звука используется PWM на TIM2 (Channel 1, PA5). Частота ноты задаётся через делитель `PSC`, громкость — через скважность (`CCR1`). Формула: $f = \texttt{TIMCLK}/((\texttt{PSC}+1)\cdot(\texttt{ARR}+1))$.

3.6 Синхронизация и защита от гонок

- `gameMutex` — мьютекс, защищающий глобальное состояние `g_game`. Используется задачами Task_Game, Task_Display и Task_Sound при чтении/записи игрового состояния. Предотвращает ситуацию, когда одна задача модифицирует состояние в то время, как другая его читает.

- `i2cMutex` — мьютекс, защищающий шину I2C1. Используется задачами Task_Input (опрос клавиатуры) и Task_Display (отправка кадра на OLED). I2C не реентерабелен, поэтому требуется мьютекс для исключения одновременных транзакций.
- `inputQueue` — очередь сообщений (FIFO, ёмкость 8, тип `uint16_t`) для передачи событий от Task_Input к Task_Game. Обеспечивает асинхронную доставку нажатий клавиш без потерь.
- `soundQueue` — очередь сообщений (FIFO, ёмкость 8, тип `uint16_t`) для передачи событий `SoundEvt_t` от Task_Game к Task_Sound. Позволяет накапливать несколько звуковых событий.

3.7 Конфигурация FreeRTOS

Параметр	**Значение**
Планирование	Вытесняющее с квантованием времени
configMAX_PRIORITIES	7
configTICK_RATE_HZ	1000 Гц
configTOTAL_HEAP_SIZE	20480 байт
configMINIMAL_STACK_SIZE	128 слов
configUSE_MUTEXES	1
configUSE_PREEMPTION	1
configUSE_TIME_SLICING	1
configGENERATE_RUN_TIME_STATS	1

Table 1: Параметры FreeRTOSConfig.h

4 Руководство пользователя

4.1 Подключение стенда

1. Подключить USB-кабель в разъём `DBG USB` стенда SDK-1.1M.
2. Убедиться, что загорелся светодиод `VTGT`.
3. В Linux проверить порты: `ls /dev/ttyUSB*` — должны появиться `ttyUSB1` и `ttyUSB2`.

4.2 Загрузка программы

1. Открыть проект `FlappyBird_FreeRTOS` в STM32CubeIDE (`File` → `Open Projects from File System`).
2. `Project` → `Build All` (Ctrl+B). Должно быть `0 errors`.
3. `Run` → `Debug`. IDE подключится к стенду через OpenOCD.
4. Нажать `F8` (Resume) для запуска.

4.3 Управление

Действие	Кнопка
Старт игры (из Ready)	Любая клавиша
Взмах (в игре)	Любая клавиша
Рестарт (после Game Over)	Любая клавиша

Table 2: Управление игрой

4.4 Игровой процесс

Режимы игры:

- **Ready** — на экране надпись «FLAPPY» и «press key». Ожидание нажатия для старта.
- **Playing** — птица летит, трубы движутся справа налево. Каждое нажатие подбрасывает птицу вверх. При отсутствии нажатий птица падает под действием гравитации. Счёт увеличивается при прохождении трубы.
- **Game Over** — столкновение с трубой или землёй. На экране надпись «OVER» и лучший результат. Нажатие перезапускает игру.

Параметры игры:

- Размер экрана: 128×64 пикселя.
- Размер птицы: 6×6 пикселей.
- Зазор между трубами: 22 пикселя.
- Ширина трубы: 14 пикселей.
- Скорость движения труб: 2 пикселя/тик.
- Гравитация: 18/100 пикселей/тик (фиксированная точка).
- Скорость взмаха: -90 (вверх).

4.5 Скриншоты

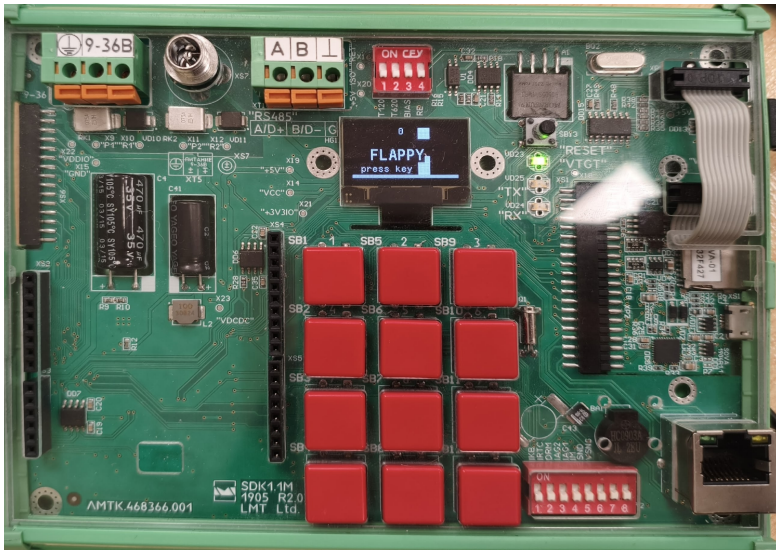


Figure 2: Стартовый экран: надпись «FLAPPY» и «press key»

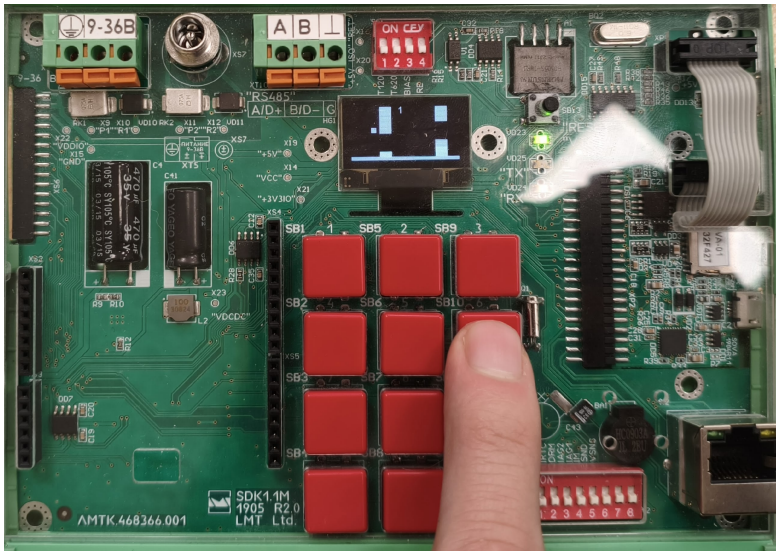


Figure 3: Игровой процесс: птица, трубы, счёт

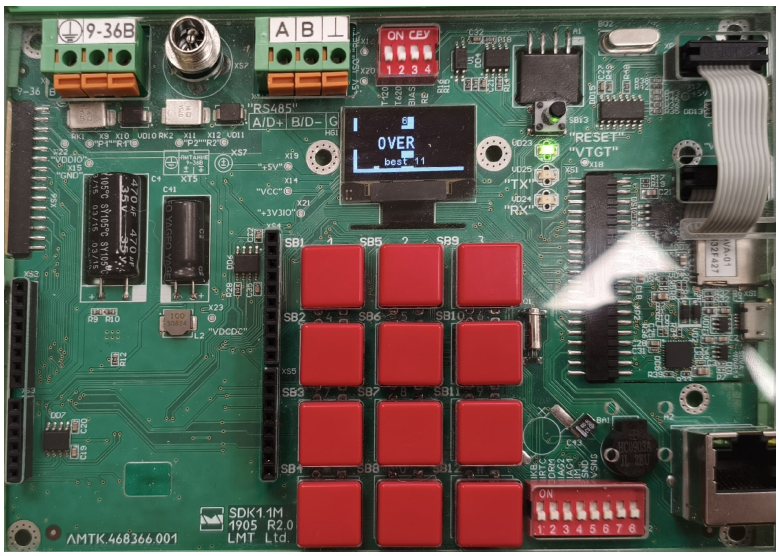


Figure 4: Экран Game Over: надпись «OVER» и лучший результат

4.6 Тестовые сценарии

Сценарий	Ожидаемый результат
Старт игры (нажать любую кнопку из Ready)	Птица подлетает, трубы начинают движение
Пролететь через первую трубу	Счёт увеличивается на 1, звуковой сигнал
Столкнуться с трубой	Игра останавливается, экран «OVER», лучший сч
Упасть на землю (не нажимать кнопки)	Game Over при касании земли
Нажать кнопку после Game Over	Новая игра, счёт сброшен, лучший счёт сохранён

Table 3: Тестовые сценарии

5 Ссылка на репозиторий с кодом

Ссылка на репозиторий: github.com/yaart/ITMO → ВСТРОЕННЫЕ СИСТЕМЫ / МК3